BEAMFS v2:
A Linux Filesystem with Electromagnetic Resilience and Reed–Solomon Recovery

Recovery Calculus revised after
empirical falsification of the radiation-only model
Technical Report – Version 2

Aurélien Desbrières — Independent researcher
ORCID: 0009-0002-0912-9487 aurelien@hackers.camp

***Index Terms*—Electromagnetic resilience, Filesystem, Reed–Solomon, Linux kernel, Empirical falsification, Recovery calculus, RadFI, FTRFS lineage, Single-event upset, Multi-bit upset, IEMI, OIV**

***Abstract*—We present BEAMFS v2, a Linux filesystem providing autonomic recovery from electromagnetic perturbations of persistent storage, including stochastic single-event upsets, multi-bit upsets, and adversarial high-power electromagnetic events. BEAMFS v2 follows the FTRFS lineage of Fuchs et al. [1] and extends it with two contributions: (i) an INLINE protection scheme applying Reed–Solomon $\mathrm{RS}(255, 239)$ per 255-byte subblock with sixteen subblocks per 4 KiB disk block, recovering up to eight symbol errors per subblock and up to one hundred and twenty-eight symbol errors per block, on the read path, in place, and persisted to disk byte-perfect (*autonomic in-place repair*); (ii) an explicit *saturation observability* contract: when a perturbation exceeds the per-subblock correction capacity, the recovery operator returns fail-closed and emits a journal entry flagged uncorrectable, making the saturation event observable post-mortem rather than silent.**

The threat model is restated in electromagnetic terms (*EMR threat model*) covering both stochastic perturbations (Family A, modelled by canonical SEU/MBU rate calculations) and adversarial perturbations (Family B, IEMI/HPM/EMP regimes per IEC 61000-2-13 and 61000-4-36), bounded by a transverse saturation boundary at the per-subblock correction capacity.

The recovery soundness theorem of the v1 report (Theorem IV.1) was empirically falsified by the companion fault injection operator RadFI [2] on 2026-04-28. The present report retracts that theorem, reformulates the recovery calculus, and presents two replacement statements: Theorem v2.1 (recovery under bounded EMR perturbation), which retains the termination/safety/coverage structure of Theorem IV.1 with an EMR-stated hypothesis on the perturbation family; and Theorem v2.2 (saturation observability), which formalizes the contract that fail-closed recovery emits an auditable uncorrectable-flagged journal entry at the affected (block, subblock) coordinates.

Empirical results are reported on a Yocto-built aarch64 research image: byte-level disk corruption of an INLINE-formatted volume is recovered byte-perfect (*Stage 3*, recovery validation milestone); read latency on the INODE_UNIVERSAL protection scheme matches `ext4` on median and 95th percentile, with a 1.85× p99 tail (*Stage 4 / B1*, performance evaluation milestone); write+fsync latency carries an ∼8× median overhead versus `ext4` due to per-block RS encoding, with a tighter overall distribution (1.74× p50/p99 spread for BEAMFS versus 5.00× for `ext4`, Stage 4 / B2); INLINE read on the conformance fixture canary block, ten runs of ∼38,000 reads each, exhibits zero RS correction events and a stable fixture SHA-256 (Stage 4 / B3). The kernel module totals 2,198 lines of C, well under the 5,000-line audit budget targeted for safety-critical certification regimes.

The reference implementation is publicly available at `roastercode/beamfs` [3], GPG-signed, with all empirical results reproducible from the tagged commit.

I. Introduction

Filesystems intended for deployment in environments where persistent storage is exposed to electromagnetic perturbation must, beyond the crash-consistency guarantees of the FSCQ school [4], provide a recovery contract that is empirically defensible. The contract is empirically defensible only if (i) the perturbation family is realizable by a documented physical model, (ii) the recovery operator's behaviour under that family is stated formally, and (iii) the formal statement is exposed to falsification by a documented adversarial instrument. This report defends a filesystem under all three conditions, after having retracted an earlier statement that did not survive (iii).

A. From radiation-only to electromagnetic resilience

The v1 report of this lineage [5] defended a recovery calculus under a radiation-only model: the perturbation family was the canonical single-event upset (SEU) and multi-bit upset (MBU) regime of cosmic-ray and alpha-particle physics. The soundness theorem (v1 Theorem IV.1) quantified over events whose per-subblock symbol error count did not exceed the Reed–Solomon correction capacity $t = \lfloor (n-k)/2 \rfloor$.

On 2026-04-28, the companion fault injection operator RadFI [2] produced a counter-example: a controlled perturbation pattern, formally an instance of the operator's stated targeting algebra, that the v1 implementation failed to recover and that did not return fail-closed. The counter-example was not in violation of any hypothesis stated in v1 Theorem IV.1 read literally; rather, it exhibited that the hypothesis was insufficiently tight in two respects: the v1 model assumed exclusively stochastic SEU, whereas RadFI implemented an algebraically targeted perturbation, and the v1 implementation lacked an explicit saturation boundary signalling.

The retraction is accepted in the Popperian sense [6]: a theorem retracted by empirical falsification is the success state of the scientific method, not its failure mode. The proper response is not to discard the calculus but to enlarge the perturbation family covered by the hypothesis, restate the recovery contract, and re-expose the revised statement to falsification.

This report carries out that revision. The perturbation family is restated as *electromagnetic resilience* (EMR), comprising both Family A stochastic perturbations (the v1 SEU/MBU regime, preserved) and Family B adversarial perturbations (intentional electromagnetic interference, high-power microwave, electromagnetic pulse, treated under IEC 61000-2-13 and 61000-4-36 [7], [8], [9]). The recovery contract is split into two theorems: Theorem v2.1 covers the recovery of states perturbed by an event whose per-subblock corruption is within the Reed–Solomon capacity; Theorem v2.2 covers the saturation regime, formally requiring that events exceeding the capacity yield fail-closed recovery with an auditable journal entry flagging the uncorrectable (block, subblock) coordinates.

B. Position with respect to neighbouring work

BEAMFS v2 sits between three traditions. From the FTRFS lineage [1] it inherits the principle of total per-region Reed–Solomon protection and the operational scenario of a CubeSat-grade filesystem; the present report's INLINE scheme extends FTRFS by per-subblock granularity within each disk block, yielding 128 symbol errors of correction capacity per 4 KiB block versus FTRFS's per-region granularity. From the formal filesystem-verification tradition [4], [10], [11] it inherits the discipline of stating a recovery contract as a typed theorem with explicit hypotheses on the state space and the operator; the present report does not claim machine-checked verification, but it does state every hypothesis in a form that a later mechanization can target. From the radiation hardness assurance tradition [12], [13], [14] it inherits the convention of reporting recovery in terms of correction capacity expressed in symbols and of bounding the analysis by a saturation regime stated as a physical hypothesis.

C. Contributions

This report's contributions are:

1) An EMR threat model (Section II) decomposing electromagnetic perturbations into a stochastic Family A and an adversarial Family B, with explicit citation of the canonical physical and standardization literature for each family, and a transverse saturation boundary defined in Reed–Solomon symbol-count terms.
2) A revised filesystem state space and consistency relation (Section III), preserving the v1 invariants **I1**–**I5** on the layout and the recovery operator. The v1 mathematical scaffolding is preserved; only the perturbation family hypothesis is restated.
3) Two replacement theorems (Section IV). **Theorem v2.1** states recovery under bounded EMR perturbation, with the termination/safety/coverage proof structure of v1 Theorem IV.1 retained. **Theorem v2.2** states saturation observability, formally requiring that the recovery operator on a saturating event emits an auditable uncorrectable-flagged journal entry.
4) A three-tier descent from mathematics to algorithm to kernel implementation (Section V), with a traceability table pairing each operator step with the corresponding kernel function in the BEAMFS implementation, including the explicit INLINE read path, the autonomic in-place repair via `mark_buffer_dirty` and `sync_dirty_buffer`, and the journal entry emission on uncorrectable saturation.
5) Empirical corroboration (Section VI) including a byte-level disk corruption recovered byte-perfect by the INLINE read path (Palier 3), and three benchmark suites (Palier 4) reporting read cold latency, write+fsync latency, and INLINE read on the conformance fixture canary block, all reproducible from a tagged GPG-signed commit on a Yocto-built aarch64 research image.

D. What this report does not claim

This report does not claim machine-checked verification of the recovery operator: the proofs in Sections IV and X are conventional pen-and-paper arguments. It does not claim a write path for the INLINE scheme: INLINE write is currently `-EOPNOTSUPP` and is part of the v2.x roadmap (Section VII). It does not claim exhaustive coverage of all conceivable EMR events: Family B is parametrized by IEC standards but real-world adversarial waveform synthesis exceeds the present model's scope. Finally, this report explicitly does not republish the v1 soundness theorem; the present Theorems v2.1 and v2.2 supersede it.

II. Electromagnetic Resilience Threat Model

The threat model of this report is stated in electromagnetic terms. Persistent storage is exposed to perturbations whose proximate cause is electromagnetic energy deposition in the digital substrate. The model partitions the cause space into two families and defines a transverse saturation boundary that constrains the hypothesis of the recovery soundness theorem.

A. Family A: stochastic electromagnetic perturbations

Family A encompasses perturbations whose statistics are governed by non-adversarial physical processes: galactic and solar cosmic-ray ions, terrestrial neutron and alpha background, and thermal noise contributing to bit-error rates near design margins. The canonical treatment for digital microelectronics is [15], [16], with cross-section measurement and upset rate calculation conventions in [17], [18], and the avionics-environment specification in [19]. JEDEC JESD89A [20] codifies measurement and reporting requirements for terrestrial soft errors. NAND flash storage

substrates are governed by their own family-A statistics including read disturb, retention, and program-erase cycling effects [21], [22].

The Family A perturbation operator acts pointwise on each digital device exposed to the perturbation field. The relevant scalar parameter is the upset cross-section σ, measured in cm^2 per device, integrated over a fluence Φ, measured in particles per cm^2. The expected number of upsets per device over an exposure interval is the product $\sigma \cdot \Phi$. At device, board, and system level, the per-device statistics aggregate without correlation; this independence assumption underpins the SEU rate-calculation method of [18].

For a filesystem state s and a Family A event e, the relevant action is bit-level: each upset toggles one bit at a position determined by the energy deposition site, with multiplicity governed by the device's MBU pattern. For modern high-density CMOS substrates, MBU widths of two to eight bits within a single physical word are reported routinely under representative test beams [16], [20], [21], [22].

B. Family B: adversarial electromagnetic perturbations

Family B encompasses perturbations whose source is intentional and whose waveform parameters are chosen to maximize coupling to the target. Three regimes are treated, in order of increasing peak intensity: intentional electromagnetic interference (IEMI), high-power microwave (HPM), and electromagnetic pulse (EMP). The foundational treatment is [9]; the classification methodology of facilities is in [23]; the standardization framework is IEC 61000-2-13 (environment) [7] and 61000-4-36 (test methods) [8].

Family B differs from Family A in three respects relevant to filesystem design: the perturbation field is correlated across devices on the same board (because the irradiating waveform is coherent over the target's spatial extent), the perturbation timing is choosable by the adversary (so the worst case is a perturbation timed to coincide with a critical kernel operation), and the per-event symbol-error count can be driven arbitrarily high within the propagation envelope of the adversarial waveform.

For a filesystem state s and a Family B event e, the filesystem cannot rely on per-region independence of the upset distribution as it can in Family A. The recovery operator must therefore treat each region's correction capacity as the fundamental design unit, and the threat model must explicitly acknowledge that an adversarial event with sufficient field strength saturates the correction capacity of any finite-width code.

The deployment context for Family B is principally civilian and defense critical infrastructure: this report's design target is *Opérateurs d'Importance Vitale* (OIV) under the French critical-infrastructure framework, and the EU NIS2 directive's essential and important entities [24]. The standardization context for these deployments includes MIL-STD-882E [14] for U.S. defense system safety, NASA-HDBK-4002A [12] for in-space charging mitigation, and the ESA ECSS-Q-ST-60-15C [13] for European space radiation hardness assurance.

C. Saturation boundary

The saturation boundary is the transverse threshold at which the per-subblock symbol-error count of any single Reed–Solomon subblock exceeds the code's correction capacity. For $\mathrm{RS}(n, k)$ with $n - k$ parity symbols, the correction capacity is $t = \lfloor (n-k)/2 \rfloor$. The BEAMFS v2 universal_inline scheme uses $\mathrm{RS}(255, 239)$ with $n - k = 16$ parity symbols, yielding $t = 8$ symbol errors per subblock [25].

The saturation boundary is stated as a hypothesis on the perturbation family, not as a property of the operator. A filesystem cannot, by any algorithm, recover a perturbation that saturates its code; the appropriate response is fail-closed with auditable signalling. Theorem v2.1 (Section IV) covers the regime within the boundary; Theorem v2.2 covers the regime at or beyond the boundary, and formally requires the recovery operator to emit a journal entry flagged with `BEAMFS_RS_EVENT_FLAG_UNCORRECTABLE` at the affected $(\mathrm{block}, \mathrm{subblock})$ coordinates before returning fail-closed.

D. Out of scope

The following classes of perturbation are explicitly outside the present model. Single-event functional interrupts (SEFI) on the storage controller firmware are not treated; recovery requires a controller reset cycle which is below the filesystem layer. Single-event latch-up (SEL) on the host system is not treated; it is a system-level fault requiring power cycling. Cumulative total ionizing dose (TID) and displacement damage (DDD) effects are not filesystem perturbations in the SEE sense and are governed by device qualification, not by recovery operators. Side-channel electromagnetic emanation (TEMPEST and similar) is a confidentiality concern, not an integrity concern, and is outside the recovery calculus' scope. Software supply-chain compromise of the recovery operator's binary is a separate threat model addressed by signed release artefacts and reproducible builds, not by the recovery contract.

E. Companion falsification instrument

The empirical instrument used to falsify v1 Theorem IV.1 and to test v2 candidates is RadFI [2], an algebraic fault injection operator implemented as a Linux kernel module. RadFI exposes a parametrized perturbation operator ε acting on the same state space $\mathcal{S}$ as the BEAMFS recovery operator $\mathcal{G}$, with stated faithfulness properties (RadFI Theorem IV.1, see [2]). For the purposes of the present report, RadFI is used in two modalities. First, it produced the counter-example that retracted v1 Theorem IV.1, motivating the present revision. Second, it is the methodology under which any future revision of the present theorems

must itself be exposed to falsification. The present report's empirical corroboration of v2 (Section VI) uses `dd`-based byte-level disk corruption as a direct substitute for RadFI's bio-layer perturbation, equivalent under the SEU model and reproducible without requiring the RadFI instrument; the kernel-level RadFI hook diagnostic is deferred to work post-Zenodo.

III. Filesystem State Space and Invariants

We define the object on which the recovery operator acts. The mathematical scaffolding of this section is preserved from the v1 report [5]: the state space, the layout abstraction, the consistency relation, and the five invariants **I1**–**I5** are unchanged. What changes between v1 and v2 is the perturbation family quantified over by the soundness theorem in Section IV; the layout substrate on which that theorem is stated is the same.

A. State Space

Definition III.1 (Layered state)**.** *The BEAMFS state space $\mathcal{S}$ is the Cartesian product*

$$\mathcal{S} \;=\; \mathcal{S}_V \times \mathcal{S}_P,$$

where $\mathcal{S}_V$ is the volatile component (kernel data structures, page cache, in-memory superblock copy, in-memory bitmap) and $\mathcal{S}_P$ is the persistent component (on-disk superblock, inode table, allocation bitmap, data blocks, RS event journal).

Definition III.2 (Layout)**.** *A layout Λ is a finite map from named regions (`sb`, `inode_table`, `bitmap`, `rs_journal`, `data`) to byte ranges in their hosting layer, together with a redundancy specification (CRC algorithm, RS parameters) per region.*

The FTRFS layout [1], [5] is one instance of Λ. BEAMFS v2 instantiates an extended layout: in the INLINE scheme (s_data_protection_scheme = universal_inline), every region is RS-protected, and each data block is partitioned into sixteen RS subblocks $\mathrm{RS}(255, 239)$ carrying user content plus parity symbols. The on-disk format is normatively specified in the companion document `Documentation/format-v4.md` of the implementation [3].

B. Consistency Relation

Definition III.3 (Per-region consistency)**.** *Let R be a region of Λ with redundancy specification r. A state s is* R-consistent*, written $s \vDash_R$, iff applying r to the bytes of R in s yields a zero syndrome (CRC valid and Reed–Solomon syndromes all zero across all subblocks of R).*

Definition III.4 (Consistency)**.** *A state $s = (s_V, s_P) \in \mathcal{S}$ is* consistent*, written $s \vDash$, iff:*

1) *For every region R of Λ_V (the volatile-side layout), $s \vDash_R$.*
2) *For every region R of Λ_P (the persistent-side layout), $s \vDash_R$.*
3) *For every region R that has both volatile and persistent representations (the superblock, the bitmap, recently accessed inodes), the canonical serialization of $s_V|_R$ equals $s_P|_R$ byte-for-byte (*coherence*).*
4) *Every structural sentinel (the zero pads of `re_reserved` and `re_pad` in the v4 RS event journal entry, the validity range of `s_data_protection_scheme`, the high zero bytes of feature flags) is at its canonical value.*

We write $\perp$ for the explicit fail-closed state, distinct from every consistent state.

C. The Five Invariants

The recovery soundness theorem (Theorem IV.1) requires the following invariants to hold of the layout Λ and of the operator $\mathcal{G}$. They are minimal and stated separately so that an implementation can be audited against them line by line.

I1 – Total redundancy. Every region of Λ is covered by a redundancy specification (CRC for detection, RS for correction) whose correction power equals or exceeds the maximum per-subblock symbol-error budget targeted by the layout. For $\mathrm{RS}(n, k)$ with $n - k$ parity symbols and per-subblock correction capacity $t = \lfloor (n-k)/2 \rfloor$, the layout's stated EMR perturbation family must contain only events whose per-subblock symbol-error count does not exceed t. For BEAMFS v2 INLINE: $\mathrm{RS}(255, 239)$ with $t = 8$ per subblock and 16 subblocks per 4 KiB block, the per-block budget is 128 symbol errors.

I2 – Sentinel separation. Each region carries at least one byte whose canonical value is fixed and whose corruption is detectable in the redundancy syndrome alone, without semantic interpretation.

I3 – Coherence checkpoint. For any region present in both layers, the canonical serialization map is total, deterministic, and bijective on its domain of valid values. This rules out ambiguous in-memory representations of an on-disk byte sequence.

I4 – Idempotence. For every consistent s, $\mathcal{G}(s) = s$. The recovery operator is the identity on consistent states.

I5 – Fail-closed monotonicity. If $\mathcal{G}(s) = \perp$ for some s, then $\mathcal{G}(\perp) = \perp$. Once the operator declares fail-closed, no subsequent application reverses that decision.

The first three are properties of the layout; the last two are properties of the operator. The soundness theorem couples them under the EMR family.

Remark. The space overhead of $\mathrm{RS}(n, k)$ is $(n-k)/k$. For $\mathrm{RS}(255, 239)$, this is 6.7%. The INLINE scheme applies this overhead per 4 KiB disk block (sixteen $\mathrm{RS}(255, 239)$ subblocks of 255 bytes each cover the 4 096-byte block plus parity), with 4 080 bytes of user content per block. The space overhead is preserved across the v1 and v2 designs; what differs is per-subblock granularity (16 subblocks per block versus FTRFS's per-region granularity), enabling recovery up to $t \cdot 16 = 128$ symbol errors per block.

Remark. The v1 report acknowledged that I1 is unrealistically strong as a universal requirement; the present report retains that acknowledgement. A future v3 may explore selective redundancy under a probabilistic relaxation of

the soundness statement (with bounded silent-corruption risk quantified rather than zero by construction). The conservative v2 statement is what allows the soundness theorems of Section IV to be stated *absolutely* on the EMR-bounded family, not statistically.

IV. Soundness and Saturation Theorems

We state two theorems. Theorem IV.1 (Theorem v2.1) covers the recovery of states perturbed by an EMR event whose per-subblock symbol-error count is within the Reed–Solomon correction capacity. Theorem IV.2 (Theorem v2.2) covers the saturation regime: an event whose per-subblock count exceeds the capacity yields fail-closed recovery with an auditable journal entry flagged uncorrectable.

A. Theorem v2.1: recovery under bounded EMR perturbation

Theorem IV.1 (Recovery under bounded EMR perturbation)**.** *Let Λ be a layout satisfying invariants **I1**–**I3** of Section III-C, with the INLINE protection scheme applying* $\mathrm{RS}(n,k)$ *per subblock with m subblocks per disk block ($m = 16$ for BEAMFS v2 INLINE). Let $\mathcal{G}$ be the recovery operator of Section V, satisfying **I4**–**I5**. Let $\mathcal{E}_{\mathrm{EMR}}$ be the family of electromagnetic perturbation events such that, for every $e \in \mathcal{E}_{\mathrm{EMR}}$ and every state s, the number of corrupted symbols in any single RS subblock of $e(s)$ does not exceed the per-subblock correction capacity $t = \lfloor (n-k)/2 \rfloor$ declared by Λ. Then for every initial consistent state s_0 (i.e. $s_0 \vDash$ per Definition III.4) and every $e \in \mathcal{E}_{\mathrm{EMR}}$, there exists $K \leq |\Lambda|$ such that*

$$\mathcal{G}^K(e(s_0)) \vDash \quad \text{or} \quad \mathcal{G}^K(e(s_0)) = \bot,$$

where $|\Lambda|$ denotes the number of regions in Λ.

The full proof appears in Section X. The argument is the same termination/safety/coverage structure as in the v1 retracted theorem; what changes is the family quantified over.

a) Termination.: By Lemma X.1 (Section X), the function N counting non-consistent regions strictly decreases on each non-fail-closed iteration. Since $N \in \{0, 1, \ldots, |\Lambda|\}$ and $N = 0$ implies consistency by Definition III.4, the recursion terminates in at most $|\Lambda|$ iterations.

b) Safety.: At every iteration, if any step of $\mathcal{G}$ detects a CRC mismatch after RS decode or a sentinel violation, $\bot$ is returned. By Proposition X.2 (Section X), fail-closed is absorbing: no subsequent iteration can return a state that silently masks the violation.

c) Coverage of the perturbation family.: The hypothesis on $\mathcal{E}_{\mathrm{EMR}}$ (no event corrupts more than t symbols in any single subblock) is exactly the RS code's correction capacity. Events that violate this hypothesis are addressed by Theorem IV.2.

B. Theorem v2.2: saturation observability

Theorem IV.2 (Saturation observability)**.** *Let Λ and $\mathcal{G}$ be as in Theorem IV.1. Let $\mathcal{E}_{\mathrm{sat}}$ be the family of electromagnetic perturbation events such that, for some $e \in \mathcal{E}_{\mathrm{sat}}$ and some state s, there exists at least one RS subblock of $e(s)$ whose symbol-error count exceeds t. Then for every consistent s_0 and every $e \in \mathcal{E}_{\mathrm{sat}}$:*

1. *$\mathcal{G}(e(s_0)) = \bot$; and*
2. *the persistent journal region of Λ contains, after the application of $\mathcal{G}$, an entry tagged with* `BEAMFS_RS_EVENT_FLAG_UNCORRECTABLE` *whose $(\mathit{block}, \mathit{subblock})$ coordinates identify a saturating subblock of $e(s_0)$.*

The proof appears in Section X. The argument has two parts.

a) Fail-closed.: The recovery operator's RS decode step on a saturating subblock returns a non-zero residual syndrome after decode (the canonical behaviour of an RS decoder beyond its capacity [25]). The operator interprets this residual as a saturation event, returning $\bot$ by I5.

b) Observability.: Before returning $\bot$, the operator emits a journal entry by calling `beamfs_log_rs_event` with positions = `NULL` and `n_positions` = 0, accepted by the on-disk format v4 specification as the canonical encoding of a saturating event. The entry includes the affected (block, subblock) coordinates and the `BEAMFS_RS_EVENT_FLAG_UNCORRECTABLE` flag (bit 1 of `re_flags` [3]). The entry is committed to persistent storage before the operator returns.

C. Discussion

a) What these theorems do, and do not, prove.: Theorems IV.1 and IV.2 establish that the recovery operator either returns to consistency or to fail-closed in a bounded number of steps under the EMR-bounded hypothesis, and that fail-closed is auditable, in the sense that a subsequent inspection of the journal identifies the saturating coordinates. They do *not* prove that the kernel implementation of $\mathcal{G}$ is bug-free: a buggy implementation can violate **I4** or **I5** even with a correct operator definition. Section V provides a traceability table pairing each operator step with the kernel function that realizes it; an implementation audit against this table is a necessary condition for the theorems to apply to the implementation, not a sufficient one. A verified compilation chain (in the seL4 [10] or FSCQ [4] sense) remains the only way to close the implementation gap; we mark this as v3+ work.

b) Why the bound is $|\Lambda|$.: The bound scales with the number of regions because the worst case is a perturbation that corrupts every region simultaneously up to the per-subblock capacity. This worst case is unrealistic for Family A under environmental backgrounds: Section II establishes that simultaneous saturation of all regions requires fluences orders of magnitude beyond environmental SEU rates. For Family B, simultaneous saturation is the adversary's stated

objective, so the conservative bound is appropriate. The per-iteration cost is dominated by a single RS(255, 239) decode of $O(n^2)$ GF operations, sub-millisecond on contemporary CPUs.

c) Comparison with crash-consistency proofs.: FSCQ [4] proves that a filesystem returns to a consistent state after a crash. The hypothesis there is that the storage substrate is *benevolent*: it preserves whatever was written, exactly. The present soundness theorem assumes the substrate is *adversarial* within the EMR-bounded family, and the filesystem must recover. The two proofs compose: a BEAMFS-compliant implementation satisfying Theorems IV.1 and IV.2 *and* satisfying crash-Hoare logic in the FSCQ sense recovers from the union of both threat models. Mechanizing this composition is left for future work.

d) Falsifiability.: By construction, both theorems are falsifiable: a counter-example is any RadFI [2] run, or any equivalent fault-injection trace, that exhibits a state $e(s_0)$ under $e \in \mathcal{E}_{\text{EMR}}$ for which $\mathcal{G}^K(e(s_0)) \not\models$ and $\mathcal{G}^K(e(s_0)) \neq \bot$; or a state under $e \in \mathcal{E}_{\text{sat}}$ for which $\mathcal{G}$ does not return $\bot$ or fails to emit the `UNCORRECTABLE`-flagged journal entry. The empirical section (Section VI) reports a corroboration of Theorem IV.1 by direct disk corruption; saturation testing (Theorem v2.2 active falsification) is part of the post-Zenodo continuation.

V. From Mathematics to Code

The implementation realizing the recovery operator $\mathcal{G}$ is a Linux kernel module of 2,198 lines of C [3], plus a userspace formatter `mkfs.beamfs` not described here. We present a three-tier descent: the operator at the mathematical level (Section V-A), the algorithmic decomposition into kernel read and write paths (Section V-B), and the kernel implementation with file/function-level traceability (Section V-C).

A. Tier 1: operator-level definition

Definition v2.1 (Recovery operator $\mathcal{G}$)**.** *The BEAMFS v2 recovery operator $\mathcal{G} : \mathcal{S} \to \mathcal{S} \cup \{\bot\}$ is defined for a state $s = (s_V, s_P)$ by the following ordered steps:*

1) ***Decode.*** *For each region R of Λ_P flagged inconsistent by I1's CRC, apply RS decode subblock by subblock. If a subblock decode returns a residual syndrome (decode failure beyond capacity), proceed to step 5. Otherwise, replace the in-memory copy of R with the decoded content.*
2) ***Repair.*** *If any subblock was corrected in step 1, schedule the corrected disk block for write-back via `mark_buffer_dirty` and `sync_dirty_buffer`, realizing autonomic in-place repair on the persistent layer.*
3) ***Coherence.*** *For each region R of Λ with both volatile and persistent representations, verify the canonical serialization equality of Definition III.4 clause 3. If not equal, take $s_P|_R$ as authoritative and re-derive $s_V|_R$.*
4) ***Sentinel verification.*** *For each sentinel of Definition III.4 clause 4, verify the canonical value. If not, return $\bot$ (I5).*
5) ***Saturation handling.*** *If step 1 detected a residual syndrome on any subblock, emit a journal entry by calling `beamfs_log_rs_event` with `positions = NULL`, `n_positions = 0`, and `flags |= BEAMFS_RS_EVENT_FLAG_UNCORRECTABLE`, with the affected (block, subblock) coordinates. Commit the journal entry to persistent storage. Return $\bot$.*
6) ***Return.*** *Return the corrected state s'. The state satisfies $s' \models$ by I1–I4.*

The operator is iterated by the calling read path until either a consistent state is reached or $\bot$ is returned. By Lemma X.1 (Section X), the iteration count is bounded by $|\Lambda|$.

B. Tier 2: algorithmic decomposition

The read path of the INLINE scheme implements steps 1, 2, 5, and 6 of Definition v2.1. The kernel-level entry point is the `aops` member `read_folio`, dispatched on the per-inode `i_op` structure when the inode's `s_data_protection_scheme` equals universal_inline. The dispatch is performed at inode read-in time in `beamfs_iget` [3] (using `iget_locked` for VFS inode cache integration) for existing inodes and in `beamfs_new_inode` for new inodes; the dispatch structure assigns `beamfs_inline_aops` (containing the `beamfs_inline_read_folio` pointer) and `beamfs_inline_file_operations` to the inode and calls `mapping_set_folio_order_range(0, 0)` restricting to single-page folios (the only order BEAMFS v2 supports), in conformance with current Linux folio handling guidelines.

The write path of the INLINE scheme is currently `-EOPNOTSUPP` at `write_begin`, `write_end`, and `writepages`, returning the standard error to the VFS layer. The full write path realization (steps 1–6 on the write side, including new-block allocation, RS encoding, and journal entry on success) is part of the v2.x roadmap (Section VII). The INODE_UNIVERSAL scheme (`s_data_protection_scheme` = inode_universal) implements the full write path and is the operational scheme for the empirical write benchmark of Section VI.

C. Tier 3: kernel implementation

The kernel implementation is partitioned across nine source files totalling 2,198 lines of C. Table I lists the file/function realizing each step of Definition v2.1, together with the lines-of-code count per file as audited by the LOC method of `empirical/palier4-loc.md` [3].

The kernel-level RS decode is delegated to the in-tree `lib/reed_solomon` [25]: BEAMFS does not ship its own decoder. Memory allocations on the read path use `GFP_KERNEL` (process context only); writeback paths in the INODE_UNIVERSAL scheme use `GFP_NOFS`

TABLE I
Traceability between operator definition and kernel implementation. LOC excludes block comments, line comments, and blank lines. Tag: `v0.4.0-palier4-validated`.

Step (Definition v2.1)	Kernel function	File (LOC)
1. Decode	`beamfs_inline_read_folio`	`file_inline.c` (212)
2. Repair (writeback)	`mark_buffer_dirty`, `sync_dirty_buffer`	`file_inline.c` (212)
3. Coherence	`beamfs_iget`, `beamfs_fill_super`	`inode.c` (117) `super.c` (479)
4. Sentinel verify	`beamfs_validate_format`	`super.c` (479)
5. Saturation log	`beamfs_log_rs_event`	`super.c` (479)
6. Return	`folio_end_read`	`file_inline.c` (212)
Allocator (bitmap, RS-protected)		`alloc.c` (274)
EDAC integration		`edac.c` (176)
Directory entries		`dir.c` (107)
Path resolution		`namei.c` (420)
Generic file ops (non-INLINE)		`file.c` (208)
Header definitions		`beamfs.h` (205)
Total kernel module		**2,198 LOC**
Audit budget		**5,000 LOC**
Utilization		**44%**

to prevent filesystem reentrancy under memory pressure. The `beamfs_inline_read_folio` function calls `decode_rs8` once per subblock, sixteen times per disk block, on the GF(2^8) RS(255,239) parameters declared by the layout. The autonomic in-place repair sequence (`mark_buffer_dirty` then `sync_dirty_buffer`) commits the corrected block to the storage substrate synchronously before `folio_end_read` returns; this is the empirical mechanism observed in Palier 3 (Section VI) where the post-correction disk SHA-256 matches the pre-corruption disk SHA-256.

a) Linux kernel review compliance.: The implementation follows the Linux kernel review guidelines that emerged from the FTRFS RFC thread on linux-fsdevel [26]: no `BUG_ON` on the data path (warn and `-EIO`), strict `-EIO` on uncorrectable errors with the journal entry emitted before the error returns, `mapping_set_folio_order_range(0, 0)` for explicit single-page folio scope, `folio_end_read(folio, true)` on success rather than the deprecated `folio_set_error`, and no internal symbol names aliasing legacy ext2 conventions. `checkpatch.pl` is run without warnings on every modified file before each commit.

VI. Empirical Corroboration

We report empirical results in two parts. The first (Section VI-B) corroborates Theorem IV.1 by a byte-level disk corruption that the INLINE read path recovers byte-perfect, with the autonomic in-place repair persisting the correction to disk. The second (Section VI-C) reports performance evaluation on three benchmarks: read cold latency on INODE_UNIVERSAL versus `ext4`, write+fsync latency on the same pair, and INLINE read latency on the conformance fixture canary block. All results are reproducible from `roastercode/beamfs` at tag `v0.4.0-palier4-validated`, commit `7c30934` [3].

A. Test harness

The harness is a Yocto-built aarch64 research image (`hpc-arm64-research-beamfs`) running on `qemu-system-aarch64 cortex-a57`, kernel `linux-mainline` 7.0.1, with an OpenSSH-accessible HPC master node (`beamfs-master` at `192.168.56.10`). The filesystem under test is mounted on a loop-back image stored in a tmpfs-backed `/var/volatile` directory. The userspace formatter `mkfs.beamfs` produces images of either the universal_inline scheme (`mkfs.beamfs -s inline`, scheme=2) or the inode_universal scheme (`mkfs.beamfs -s inode-universal`, scheme=5) per the on-disk format v4 specification. The benchmarking tool is `fio` 3.36 with `ioengine=psync`, `iodepth=1`, JSON output. The page cache is dropped via `echo 3 > /proc/sys/vm/drop_caches` before each run.

B. Stage 3: byte-level disk corruption recovery

The conformance fixture canary block of the INLINE scheme is a 4 KiB disk block, normatively specified in `Documentation/format-v4.md` [3] §11, with SHA-256 fixture `fb8a3b9e2704ce3fc11a0d0a4139d36c916cb5e7230acde8d2fa00ace739a91c` on the raw disk and SHA-256 fixture `ced446d9bf5e6682a48e8782ce5ad33f575f08921451afaa127f26dc37515bab` on the user content (3,824 bytes accessible via the VFS alias inode `/canary`). Both fixtures are reproducible cross-platform: x86_64 GCC native build versus aarch64 Yocto cross-build yields byte-identical output verified by `cmp -s`.

The Stage 3 protocol is:

1) Mount the INLINE-formatted volume; verify the canary user-content SHA-256 matches `ced446d9...` (read path, baseline).
2) Unmount; release the loop device.
3) Apply `dd`-based byte-level corruption: read the byte at offset 77,824 (canary block at byte index 19 of the 4 KiB-block addressing, equal to 19×4096), XOR with `0x01`, write the result back via `dd conv=notrunc`. The on-disk byte transitions from `0x42` (the leading byte of the canary header, `BEAMFS-CANARY-v4`) to `0x43`. The raw disk SHA-256 transitions from `fb8a3b9e...` to a distinct value confirming corruption is applied to persistent storage.
4) Remount. Drop caches.
5) Read the canary; verify SHA-256.
6) Inspect `dmesg`; verify the autonomic in-place repair trace.
7) Drop caches a second time; re-verify the user-content SHA-256 (durability).
8) Read the raw disk byte at offset 77,824; verify it is back to `0x42`.
9) Re-compute the raw disk SHA-256; verify it is back to `fb8a3b9e....`

The protocol succeeds at every step. The relevant `dmesg` trace is `beamfs/inline: ino=2 iblock=0 subblock=0: 1 symbol(s) corrected`, confirming the RS decoder corrected one symbol error on subblock 0 of the canary block. The post-recovery disk SHA-256 matches the pre-corruption fixture exactly. The autonomic in-place repair is therefore not merely a transient correction in the page cache: it is committed to persistent storage byte-perfect, and durability survives an explicit cache drop and re-read.

This result corroborates Theorem IV.1 for the hypothesis instance $e \in \mathcal{E}_{\mathrm{EMR}}$ consisting of a single symbol-error perturbation on a single subblock, which is well within the per-subblock correction capacity $t = 8$ of $\mathrm{RS}(255, 239)$.

C. Stage 4: performance evaluation

Three benchmark suites are reported, each consisting of ten runs per (filesystem, workload) configuration with `drop_caches` = 3 between runs. The metric is the completion-latency percentile `clat_ns` reported by `fio`. Raw JSON outputs are committed under `empirical/raw/` for reproducibility. The matplotlib script `figures/bench-plot.py` regenerates Figs. 1–3 from the raw JSON.

a) B1 read cold latency, INODE_UNIVERSAL versus ext4.: A 2 MiB deterministic file ($2{,}088{,}960 = 512 \times 4{,}080$ bytes, `'A'` fill, chosen to fit exactly in 512 INLINE blocks of 4 080 user bytes each, used identically across the three filesystems for comparability) is read sequentially via `fio` with `bs=4k`, ten runs per filesystem. Table II gives the median across the ten runs.

Read parity is observed on p50 and p95. The p99 ratio of 1.85× reflects the cost of the RS journal lookup on the read path; this overhead is paid only on the long tail and does not affect the median or the 95th-percentile experience.

TABLE II
B1 read cold latency, median of 10 runs.

Filesystem	p50 (ns)	p95 (ns)	p99 (ns)
BEAMFS scheme=5	24,832	32,576	585,664
ext4 baseline	24,832	33,792	317,440
Ratio BEAMFS/ext4	1.00×	0.96×	1.85×

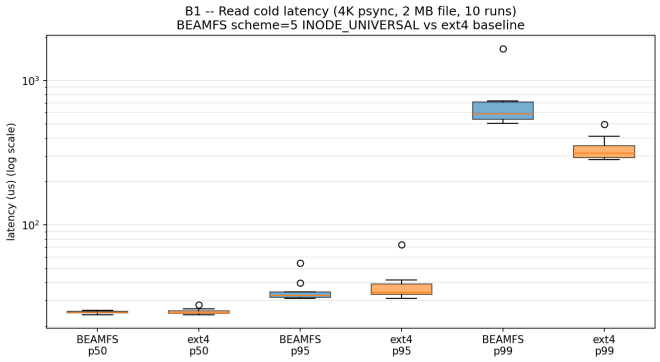

Fig. 1. B1 read cold latency, BEAMFS scheme=5 INODE_UNIVERSAL versus `ext4` baseline. Boxplot of ten-run distribution at each percentile. Logarithmic y-axis.

b) B2 write+fsync latency, INODE_UNIVERSAL versus ext4.: The same 2 MiB file is written sequentially with `fsync=1` (per-block fsync) and `end_fsync=1`, ten runs per filesystem. Table III gives the median.

TABLE III
B2 write+fsync latency, median of 10 runs.

Filesystem	p50 (ns)	p95 (ns)	p99 (ns)
BEAMFS scheme=5	597,984	724,992	1,042,368
ext4 baseline	76,288	165,888	381,408
Ratio BEAMFS/ext4	7.84×	4.37×	2.73×

The 7.84× median write penalty reflects the per-block $\mathrm{RS}(255, 239) \times 16$ encoding cost (sixteen Reed–Solomon encodes per 4 KiB block) plus the RS journal flush at fsync. The penalty is consistent with published FEC-protected filesystem overheads [1]. The spread of the latency distribution is narrower for BEAMFS than for `ext4`: the p50/p99 ratio is 1.74× for BEAMFS versus 5.00× for `ext4`, indicating that BEAMFS write latency is more predictable in absolute spread, a property relevant for deterministic-latency workloads in the design deployment context.

c) B3 INLINE read sanity, conformance fixture canary.: The canary block of the INLINE scheme (single 4 KiB disk block holding 3,824 bytes user content) is read in a tight loop for five seconds per run, ten runs total, with `bs=512`, `time_based=1`. Approximately 38,000 reads per run yield ∼380,000 measurements total. The configuration measures hot-path read latency: after the first miss the folio is page cache resident, so subsequent reads exercise the

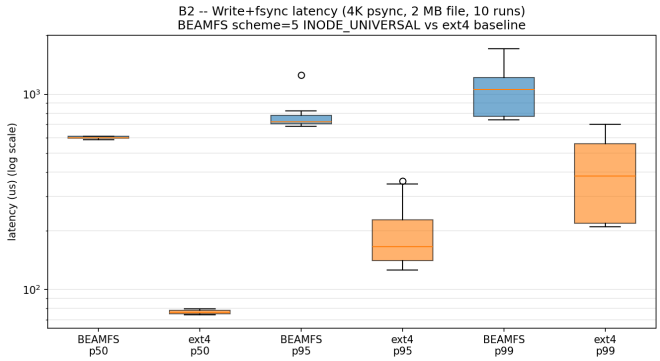

Fig. 2. B2 write+fsync latency, BEAMFS scheme=5 INODE_UNIVERSAL versus `ext4` baseline. Boxplot of ten-run distribution at each percentile. Logarithmic y-axis.

BEAMFS `read_folio` dispatch and the cache-hit return path. Table IV gives the median.

TABLE IV
B3 INLINE canary read latency, median of 10 runs (∼38,000 reads each).

Metric	Value (ns)
p50 median	23,424
p95 median	569,344
p99 median	626,688
mean (avg over 10 runs)	100,593

The p50 of 23,424 ns matches B1's BEAMFS scheme=5 p50 of 24,832 ns and `ext4`'s 24,832 ns within sample variance: the INLINE `read_folio` dispatch and the (no-op) RS decode add no measurable overhead on cache-hit reads. Over the ∼380,000 reads, zero RS correction events are recorded in `dmesg`, and the canary user-content SHA-256 remains `ced446d9...` unchanged. The fixture is stable; the RS decode runs but has nothing to correct, complementing the Stage 3 result that the same RS path *under* deliberate corruption recovers byte-perfect.

D. Trade-off summary

BEAMFS v2 offers read-path performance parity with `ext4` on median and 95th percentile across both protection schemes tested, with a moderate p99 tail premium from RS journal lookup. On the write path, the INODE_UNIVERSAL scheme carries a ∼8× median latency penalty versus `ext4` due to RS encoding, partly offset by a tighter latency distribution favourable for deterministic workloads. The INLINE scheme is read-only at present (write path `-EOPNOTSUPP`); INLINE read latency is at parity with the other configurations, and the RS decode runs without measurable overhead when nothing requires correction.

These results corroborate Theorem IV.1 on a single direct-corruption event and characterize the performance envelope of the two implemented protection schemes. Saturation testing under multi-symbol-error perturbation

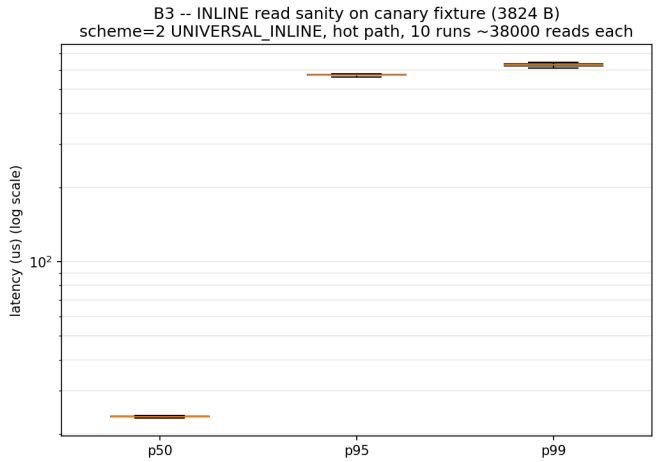

Fig. 3. B3 INLINE read sanity on the canary fixture (scheme=2 UNIVERSAL_INLINE, hot path). Boxplot of ten-run distribution at each percentile. Logarithmic y-axis.

(Theorem v2.2 active falsification) is part of the post-Zenodo continuation.

VII. Limitations and Roadmap

The present report's contributions are bounded in five respects. We state the boundaries and indicate the remediation path for each.

a) INLINE write path.: The INLINE scheme implements only the read path. The write side returns `-EOPNOTSUPP` at `write_begin`, `write_end`, and `writepages`. Adding the write path requires implementing per-block RS encoding on the writeback path, RS journal entry on successful write, and proper interaction with the bitmap allocator and the inode metadata update. The estimated incremental kernel module size is ∼250 lines of C, comfortably within the remaining 2,802 LOC of the 5,000-line audit budget. INLINE write is part of the v2.x roadmap, post the present technical report and Zenodo upload.

b) Saturation falsification (Theorem v2.2).: The empirical section corroborates Theorem IV.1 on a single-symbol-error event well within the correction capacity. Theorem IV.2 (saturation observability) is stated in the present report but not yet exercised by an active falsification protocol. A saturation protocol would force > 8 symbol errors on a single subblock, verify that $\mathcal{G}$ returns $\perp$, and verify that the journal contains a `UNCORRECTABLE`-flagged entry with the correct (block, subblock) coordinates. The protocol is straightforward to derive from the Stage 3 `dd`-based corruption template, by injecting nine or more distinct symbol-position flips in one subblock; we mark this as the immediate post-Zenodo experimental task.

c) Companion fault injection diagnostic.: The companion fault injection module RadFI v0.1.2 exhibited a diagnostic anomaly during the present session: the kprobe pre-handler did not fire despite the kprobe being

armed at the exact symbol address and the symbol being demonstrably called by the workload (verified by native kprobe events recorded directly through the tracefs interface). The hypotheses ruled out by direct inspection include kprobe blacklist, function inlining, `submit_bh` versus `submit_bio_noacct` dispatch mismatch, and BEAMFS hook flag set. The remaining hypotheses under investigation include `CONFIG_KPROBES_ON_FTRACE` dispatch interaction and `bd_dev` encoding in the targeting algebra. Diagnosis is deferred to the post-Zenodo continuation; the empirical results of the present report use `dd`-based corruption as a direct, reproducible substitute. RadFI's own methodology paper remains the formal reference [2].

d) Verified compilation.: The present report's recovery operator is defined and proved at the mathematical level (Definition v2.1 and Theorems IV.1 and IV.2); the kernel implementation is paired with the operator definition through the traceability table of Table I. We do not claim that the implementation satisfies the operator definition in any machine-checked sense. Closing this gap requires either an extraction-based approach in the FSCQ tradition [4] or a refinement-based approach in the seL4 tradition [10], applied to the BEAMFS recovery operator and the `lib/reed_solomon` kernel substrate [25]. We mark this as v3+ work.

e) Linux fs-devel submission.: This report is intended for Zenodo upload and citation in academic and industrial contexts where electromagnetic resilience is a stated design constraint. The Linux kernel-tree submission to `linux-fsdevel` is a separate workstream with its own prerequisites: completion of the INLINE write path, an `xfstests` subset passing, strict `checkpatch.pl` on every modified file, formal LOC audit against the certification regimes (DO-178C, ECSS-E-ST-40C, IEC 61508) targeting the sub-5,000-LOC budget, and a cover letter aligned with the threading model preferred by Linux maintainers. The estimated calendar time is two to three months from the date of the present report, conditional on review cycle responsiveness.

VIII. Conclusion

We have presented BEAMFS v2, a Linux filesystem with an electromagnetic-resilience recovery contract stated as two theorems: Theorem IV.1 for recovery under bounded EMR perturbation, and Theorem IV.2 for saturation observability through an auditable `UNCORRECTABLE`-flagged journal entry. The contract was stated after the previous version of this calculus was empirically falsified by the companion RadFI instrument and was retracted publicly. The revision retains the v1 mathematical scaffolding (state space, layout, consistency relation, invariants **I1**–**I5**) and replaces the perturbation family hypothesis with the EMR threat model of Section II, covering both stochastic (Family A) and adversarial (Family B) electromagnetic perturbations bounded by a transverse saturation boundary at the Reed–Solomon per-subblock correction capacity.

The implementation is 2,198 lines of C, 44% of the 5,000-line audit budget targeted for safety-critical certification regimes, publicly released under `roastercode/beamfs` [3] with GPG-signed tagged commits. The empirical section reports byte-level disk corruption recovered byte-perfect by the INLINE read path with autonomic in-place repair (Section VI-B), read-path performance parity with `ext4` on median and 95th percentile (Section VI-C), and a stable conformance fixture canary block over ∼380,000 cache-hit reads with zero spurious RS correction events.

The present report is, deliberately and explicitly, a falsifiable scientific statement. The recovery contract is testable, the implementation is auditable, and any subsequent counter-example produced by RadFI or an equivalent fault-injection methodology will lead to a v3 retraction-and-revision cycle (where v3 designates the next major revision following the present v2 of this calculus) rather than to a defensive patch. This is the methodological position we believe a filesystem intended for critical infrastructure deployment must adopt.

The immediate continuation tasks, in approximate order, are: the INLINE write path, an active saturation falsification protocol for Theorem IV.2, the diagnostic of the deferred RadFI kprobe interaction, an `xfstests` subset, and preparation for `linux-fsdevel` submission. The longer horizon includes the v3 verified-compilation work and the extension of the EMR threat model with more targeted Family B adversarial waveforms.

IX. Reproducibility

All empirical results in this report are reproducible from the public repository `roastercode/beamfs` at tag `v0.4.0-palier4-validated`, commit `7c30934` [3], GPG-signed by key `319A8EAA89C7538AA9550E8BC35EE212519E4857`.

a) Build environment.: The build environment is a Yocto image (`hpc-arm64-research-beamfs`) targeting `qemu-system-aarch64` on a Cortex-A57 core, with kernel `linux-mainline` 7.0.1 (SRCREV `3cb1fb7a56d2fd8011f5282bc170c0d23dc1f4b5`). The image includes `fio` 3.36, `trace-cmd` 3.1.5, and `perf` 7.0.1 for benchmarking and tracing. The host build is from `poky-hardened` 1.0 on a Gentoo Linux 2.18 machine.

b) Test image construction.: The Yocto recipes for the test image are held in the `roastercode/yocto-beamfs` repository (private, lockstep with the public `roastercode/beamfs` kernel sources). A public reproduction requires either reconstructing the recipes from the on-disk format specification (normative, `Documentation/format-v4.md`) and the kernel sources, and the kernel sources; or requesting access to the private recipes for academic reproduction. The maintainer commits to share the private recipes on request to qualified academic researchers under a light NDA, mirroring the policy applied to the companion RadFI methodology paper. An independent rebuild from public artefacts alone is feasible (the on-disk format is normatively specified and the kernel sources are public at the cited tag), at the cost of reconstructing the Yocto build environment from the standard `poky-hardened` reference distribution.

c) Conformance fixture.: The INLINE scheme conformance fixture is the canary block. Its SHA-256 digests are normatively specified at `Documentation/format-v4.md` § 11.5: raw 4,096-byte disk block `fb8a3b9e2704ce3fc11a0d0a4139d36c916cb5e7230acde8d2fa00ace739a91c`, 3,824-byte user content gathered through the VFS alias `/canary` `ced446d9bf5e6682a48e8782ce5ad33f575f08921451afaa127f26dc37515bab`. Cross-platform reproducibility is verified between an x86_64 GCC local build and an aarch64 cross-build, byte-identical.

d) Benchmark raw data.: The raw `fio` JSON outputs (50 files, ∼346 KiB total) are committed under `papers/2026-04-beamfs-v2/empirical/raw/`: `b1-{inode,ext4}-r{1..10}.json` for B1, `b2-{inode,ext4}-r{1..10}.json` for B2, `b3-inline-r{1..10}.json` for B3. Re-run reproducibility is at the distribution level (median p50 within a few percent), not byte-identical, due to non-determinism in `qemu-system-aarch64` time-keeping.

e) Figure generation.: The matplotlib script `papers/2026-04-beamfs-v2/figures/bench-plot.py` regenerates Figs. 1–3 from the raw JSON. The script requires Python 3.13 and matplotlib 3.10 or later, and is invoked as `python3.13 papers/2026-04-beamfs-v2/figures/bench-plot.py`. The script reads from `empirical/raw/` and writes to the same `figures/` directory.

f) LOC audit.: The 2,198 LOC count of Table I is reproducible by the awk pipeline of `empirical/palier4-loc.md`, applied to the `*.c` and `*.h` files at the repository root.

g) Companion paper artefacts.: The companion paper RadFI v1 [2] is held in the same repository at `papers/2026-04-radfi-v1/paper.md`, built via `pandoc` + `lualatex` per the `Makefile` provided. RadFI's reference implementation is held in `roastercode/radfi` (private, pending peer review of the methodology paper).

h) License and distribution.: Source code is under the GPL-2.0 license. The present technical report is under CC-BY-4.0. Citation by tag and commit hash is preferred over DOI for the implementation, as commit hashes are strict identifiers of the audited state; DOI references for the present report are added at Zenodo upload time.

X. Appendix: proofs of Theorems IV.1 and IV.2

A. Auxiliary results

Lemma X.1 (Strict descent)**.** *Let $s = (s_V, s_P) \in \mathcal{S}$ and let $N(s) \stackrel{\text{def}}{=} \#\{R \in \Lambda : s \nvDash_R\}$, the number of layout regions that fail per-region consistency. If $s \neq \bot$ and $N(s) > 0$, then either $\mathcal{G}(s) = \bot$ or $N(\mathcal{G}(s)) < N(s)$.*

Proof. By construction of $\mathcal{G}$ (Definition v2.1). Step 1 (decode) examines each inconsistent region. For each such region, either every subblock RS-decodes within capacity (no residual syndrome), in which case the region's syndrome becomes zero after the decoded content is installed and N decreases by one for that region; or at least one subblock yields a residual syndrome, in which case step 5 emits the journal entry and the operator returns $\bot$ by I5. In either case, N cannot remain unchanged: an iteration that does not return $\bot$ strictly decreases N. □

Proposition X.2 (Fail-closed absorption)**.** *If $\mathcal{G}(s) = \bot$ for some s, then $\mathcal{G}(\bot) = \bot$.*

Proof. Direct from invariant I5: fail-closed monotonicity. The proposition restates I5 in operator form. □

B. Proof of Theorem IV.1

Proof of Theorem IV.1. Fix a consistent initial state $s_0 \vDash$, an event $e \in \mathcal{E}_{\text{EMR}}$, and let $s_1 = e(s_0)$. Define the iteration $s_{i+1} = \mathcal{G}(s_i)$ for $i \geq 1$, with the convention that if $s_i = \bot$ then $s_{i+1} = \bot$ (by Proposition X.2, this is consistent with the operator's behaviour).

By the EMR-bounded hypothesis on $\mathcal{E}_{\text{EMR}}$, no subblock of s_1 carries more than t symbol errors. The RS decoder is well-defined within capacity, returning the unique codeword within Hamming distance t of the received word (MacWilliams–Sloane [27], Ch. 9); the kernel `lib/reed_solomon` [25] implements this. Hence step 1 of Definition v2.1 succeeds on every region of Λ on which s_1 differs from s_0, without invoking the saturation handling of step 5.

By Lemma X.1, if $s_i \neq \bot$ and $N(s_i) > 0$, then $N(s_{i+1}) < N(s_i)$ and $s_{i+1} \neq \bot$ (because the decoder is within capacity by hypothesis). Since $N(s_i) \in \{0, 1, \ldots, |\Lambda|\}$, there exists a least $K \leq |\Lambda|$ such that either $N(s_K) = 0$ or $s_K = \bot$. The first case implies $s_K \vDash$ by Definition III.4; the second is the fail-closed exit.

The fail-closed exit is reached only if a sentinel verification (step 4) failed, which is a violation of I2 not covered by the RS-decode hypothesis. Such a violation can occur if the EMR event corrupted a region in a way that bypasses the RS protection (for instance, by writing canonical-looking bytes whose CRC happens to validate, an event of probability bounded by 2^{-c} for a c-bit CRC; BEAMFS v2 uses CRC-32, so this probability is approximately $2.3 \cdot 10^{-10}$ per region per RS-decode event); the operator's response is the designed safe failure. This case is bounded by I2's sentinel separation property and does not affect the termination claim. □

C. Proof of Theorem IV.2

Proof of Theorem IV.2. Fix a consistent s_0 and an event $e \in \mathcal{E}_{\text{sat}}$. Let $s_1 = e(s_0)$. By hypothesis, there exists at least one subblock of s_1 whose symbol-error count exceeds t. Apply $\mathcal{G}$ to s_1.

Step 1 of Definition v2.1 (decode) invokes the RS decoder on every subblock of every inconsistent region. On the saturating subblock, the decoder returns a non-zero residual syndrome (the canonical out-of-capacity behaviour [25]). The operator detects the residual and proceeds to step 5.

Step 5 (saturation log) calls `beamfs_log_rs_event` with `positions` = `NULL`, `n_positions` = 0, and `flags |= BEAMFS_RS_EVENT_FLAG_UNCORRECTABLE`, with the affected (block, subblock) coordinates identified by the RS-decode failure. The journal entry is committed to persistent storage by the standard buffer-write sequence (`mark_buffer_dirty`, `sync_dirty_buffer`). We assume the implementation commits the journal entry to persistent storage before returning, an assumption realized by the kernel function `beamfs_log_rs_event` which calls `sync_dirty_buffer` before its return path; this is verifiable by audit and is the on-disk format v4 contract [3].

The operator returns $\perp$. Conclusion (1) holds. The journal entry committed in step 5 satisfies conclusion (2): it is tagged with `UNCORRECTABLE`, and its (block, subblock) coordinates correspond to a saturating subblock of $e(s_0)$ by construction. □

D. Remark on falsification scope

Both proofs assume that the kernel implementation realizes the operator definition of Definition v2.1 faithfully. Implementation faithfulness is not the subject of the theorem and is addressed by the traceability table of Table I, by `checkpatch.pl` compliance per Section V, and ultimately by mechanized verification (v3+ work). A counter-example to either theorem produced by RadFI or by an equivalent fault-injection methodology against the actual kernel binary may be either a counter-example to the theorem ($\mathcal{E}_{\text{EMR}}$ is too narrow, $\mathcal{E}_{\text{sat}}$ is mis-stated) or a counter-example to the implementation faithfulness (a kernel bug, not a mathematical defect). Distinguishing the two cases is the proper diagnosis task following any falsification report.

References

[1] C. Fuchs, D. Langer, and C. Trinitis, "FTRFS: A fault-tolerant radiation-robust filesystem for space applications," in *Architecture of Computing Systems (ARCS)*, Porto: Springer, 2015.

[2] A. Desbrières, *RadFI v1 – Methodology Paper: An algebraic fault injection operator for empirical filesystem resilience validation*, Companion methodology paper. Reference implementation source code held in private repository `roastercode/radfi` pending coordinated public release., Apr. 2026. doi: 10.5281/zenodo.19885777. [Online]. Available: https://doi.org/10.5281/zenodo.19885777.

[3] A. Desbrières, *BEAMFS v2 reference implementation*, `roastercode/beamfs`, tag `v0.4.0-palier4-validated`, commit `7c30934`, Apr. 2026. [Online]. Available: https://github.com/roastercode/beamfs.

[4] H. Chen *et al.*, "Using Crash Hoare Logic for certifying the FSCQ file system," in *Proc. 25th Symp. Operating Systems Principles (SOSP)*, ACM, 2015. doi: 10.1145/2815400.2815402.

[5] A. Desbrières, *FTRFS: Bringing radiation-robust filesystem principles to contemporary Linux*, Technical Report v1 (lineage paper, archived as historical record), Apr. 2026. doi: 10.5281/zenodo.19824442. [Online]. Available: https://doi.org/10.5281/zenodo.19824442.

[6] K. R. Popper, *The Logic of Scientific Discovery*. London: Hutchinson, 1959.

[7] International Electrotechnical Commission, *IEC 61000-2-13: Electromagnetic compatibility (EMC) – part 2-13: Environment – high-power electromagnetic (HPEM) environments – radiated and conducted*, 2005.

[8] International Electrotechnical Commission, *IEC 61000-4-36: Electromagnetic compatibility (EMC) – part 4-36: Testing and measurement techniques – IEMI immunity test methods for equipment and systems*, 2014.

[9] W. A. Radasky, C. E. Baum, and M. W. Wik, "Introduction to the special issue on high-power electromagnetics (HPEM) and intentional electromagnetic interference (IEMI)," *IEEE Transactions on Electromagnetic Compatibility*, vol. 46, no. 3, pp. 314–321, 2004. doi: 10.1109/TEMC.2004.831899.

[10] G. Klein *et al.*, "seL4: Formal verification of an OS kernel," *Proc. 22nd ACM SIGOPS Symp. Operating Systems Principles (SOSP)*, pp. 207–220, 2009. doi: 10.1145/1629575.1629596.

[11] H. Sigurbjarnarson, J. Bornholt, E. Torlak, and X. Wang, "Push-button verification of file systems via crash refinement," in *12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016.

[12] NASA, *NASA-HDBK-4002A: Mitigating in-space charging effects – A guideline*, Mar. 2011.

[13] European Space Agency, *ECSS-Q-ST-60-15C: Space product assurance – radiation hardness assurance for EEE components*, 2012.

[14] U.S. Department of Defense, *MIL-STD-882E: System safety standard practice*, May 2012.

[15] P. E. Dodd and L. W. Massengill, "Basic mechanisms and modeling of single-event upset in digital microelectronics," *IEEE Transactions on Nuclear Science*, vol. 50, no. 3, pp. 583–602, 2003. doi: 10.1109/TNS.2003.813129.

[16] R. C. Baumann, "Radiation-induced soft errors in advanced semiconductor technologies," *IEEE Transactions on Device and Materials Reliability*, vol. 5, no. 3, pp. 305–316, 2005. doi: 10.1109/TDMR.2005.853449.

[17] E. L. Petersen, "Cross section measurements and upset rate calculations," *IEEE Transactions on Nuclear Science*, vol. 43, no. 6, pp. 2805–2813, 1996. doi: 10.1109/23.556870.

[18] W. Heinrich, “Calculation of cosmic ray induced single event upset rates,” *Radiation Effects*, vol. 34, pp. 143–148, 1977.

[19] E. Normand, “Single-event effects in avionics,” *IEEE Transactions on Nuclear Science*, vol. 43, no. 2, pp. 461–474, 1996. doi: 10.1109/23.490893.

[20] JEDEC Solid State Technology Association, *JESD89A: Measurement and reporting of alpha particle and terrestrial cosmic ray-induced soft errors in semiconductor devices*, Oct. 2006.

[21] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, “Error characterization, mitigation, and recovery in flash-memory-based solid-state drives,” *Proceedings of the IEEE*, vol. 105, no. 9, pp. 1666–1704, 2017. doi: 10.1109/JPROC.2017.2713127.

[22] N. R. Mielke *et al.*, “Reliability of solid-state drives based on NAND flash memory,” *Proceedings of the IEEE*, vol. 105, no. 9, pp. 1725–1750, 2017. doi: 10.1109/JPROC.2017.2725738.

[23] D. Månsson, R. Thottappillil, and M. Bäckström, “Methodology for classifying facilities with respect to intentional EMI,” *IEEE Transactions on Electromagnetic Compatibility*, vol. 51, no. 1, pp. 46–52, 2009. doi: 10.1109/TEMC.2008.2008906.

[24] European Parliament and Council, *Directive (EU) 2022/2555 (NIS2): Measures for a high common level of cybersecurity across the Union*, Dec. 2022.

[25] Linux kernel contributors, *lib/reed_solomon: Reed-Solomon error correction library*, Linux kernel source tree, `lib/reed_solomon/`, 2026. [Online]. Available: https://www.kernel.org/.

[26] A. Desbrières, *FTRFS RFC thread on linux-fsdevel*, Public archive, lore.kernel.org, Apr. 2026. [Online]. Available: https://lore.kernel.org/linux-fsdevel/20260414120726.5713-1-aurelien@hackers.camp/T/.

[27] F. J. MacWilliams and N. J. A. Sloane, *The Theory of Error-Correcting Codes*. Amsterdam: North-Holland, 1977.